

VISUALIZATION

Data visualization is the graphical representation of data through charts and graphs, enabling the identification of trends, outliers, and patterns. It simplifies complex data, making it easier to understand and communicate insights effectively.

Visualization aids in uncovering relationships between variables, tracking changes over time, and highlighting key data points.

Several libraries and tools are available for creating these visualizations in Python, with some of the most popular ones being Matplotlib, Seaborn, Plotly, Pandas, and Bokeh. These libraries offer diverse features and capabilities, allowing users to create a wide range of static, interactive, and dynamic visualizations tailored to their specific needs.

Matplotlib

Library Overview

Matplotlib is a powerful, foundational library for static 2D plotting in Python. It gives users full control over every aspect of a plot, making it ideal for creating publication-quality charts and custom visualizations.

Use Cases: Academic plots, detailed control, static report visuals.

Key Features:

- Fine-grained control over plot components
- Wide variety of chart types
- Integrates with NumPy, Pandas, and more
- Export to various file formats (PDF, PNG, SVG)

Graph Types and Examples

1. Line Plot

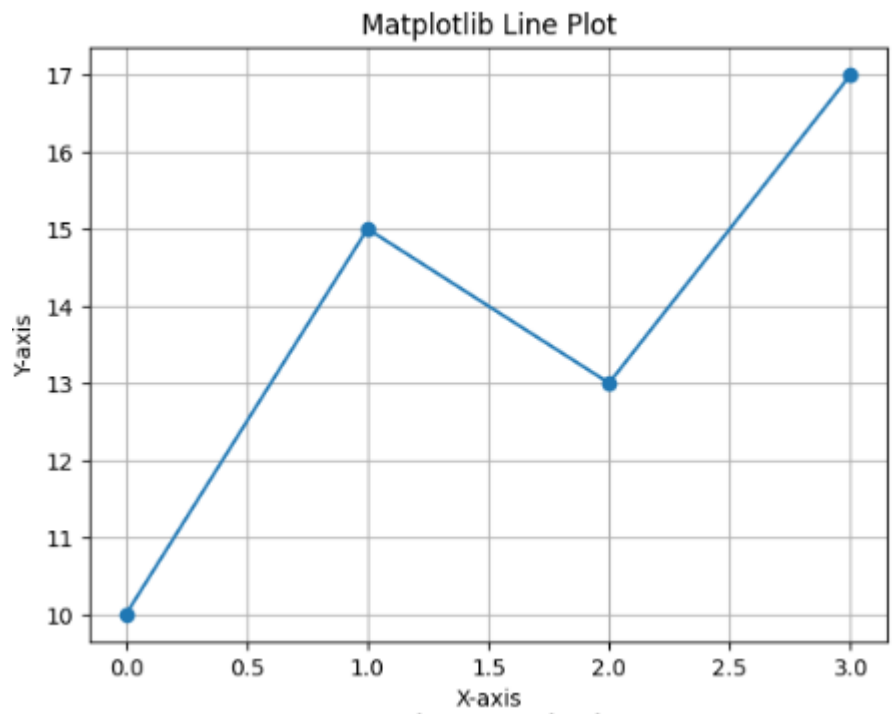
Description: A line plot connects data points with a continuous line. It's used to show trends over time or ordered categories.

Use Case: Tracking stock prices, temperature changes over time, or performance metrics.

```
import matplotlib.pyplot as plt

x = [0, 1, 2, 3]
y = [10, 15, 13, 17]

plt.plot(x, y, marker='o')
plt.title('Matplotlib Line Plot')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.grid(True)
plt.show()
```



2. Scatter Plot

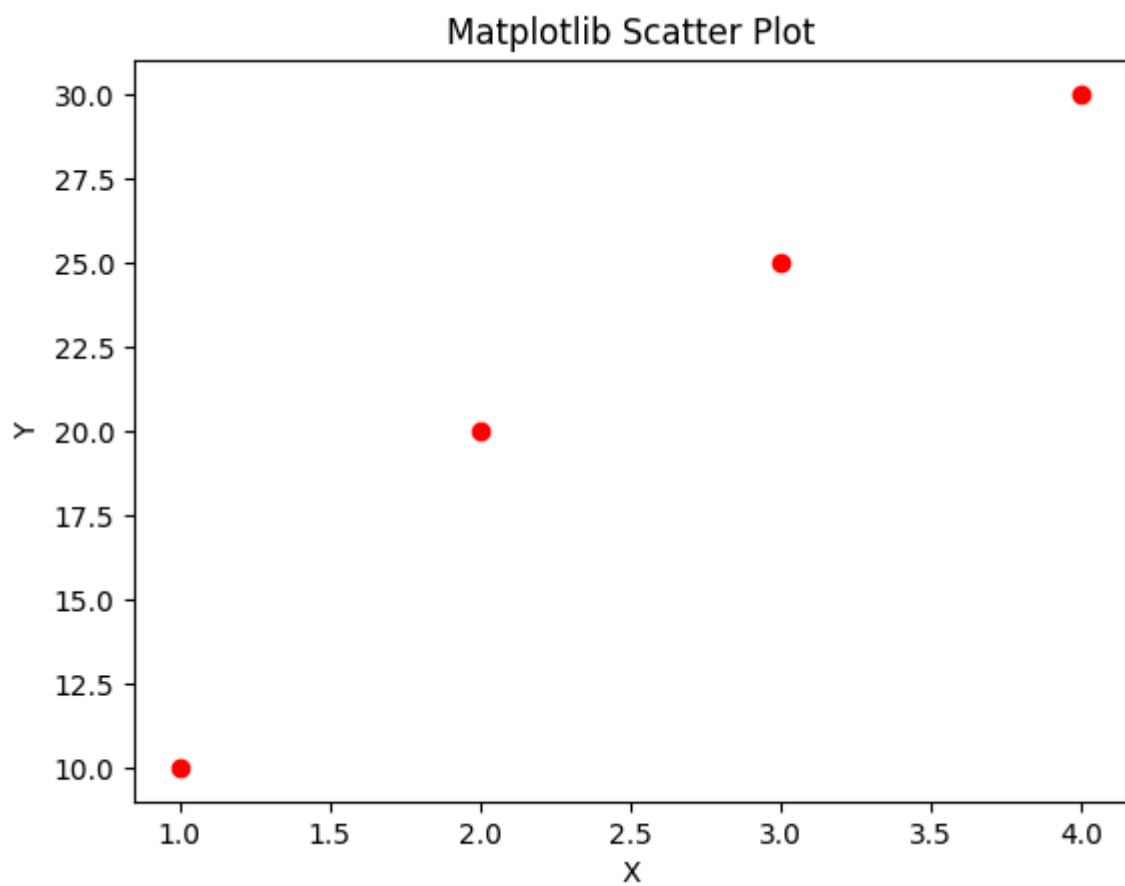
Description: A scatter plot displays points representing the values of two variables. It shows correlation, clustering, and outliers.

Use Case: Visualizing relationships between variables (e.g., height vs weight).

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]
y = [10, 20, 25, 30]

plt.scatter(x, y, color='red')
plt.title('Matplotlib Scatter Plot')
plt.xlabel('X')
plt.ylabel('Y')
plt.show()
```



3. Bar Chart

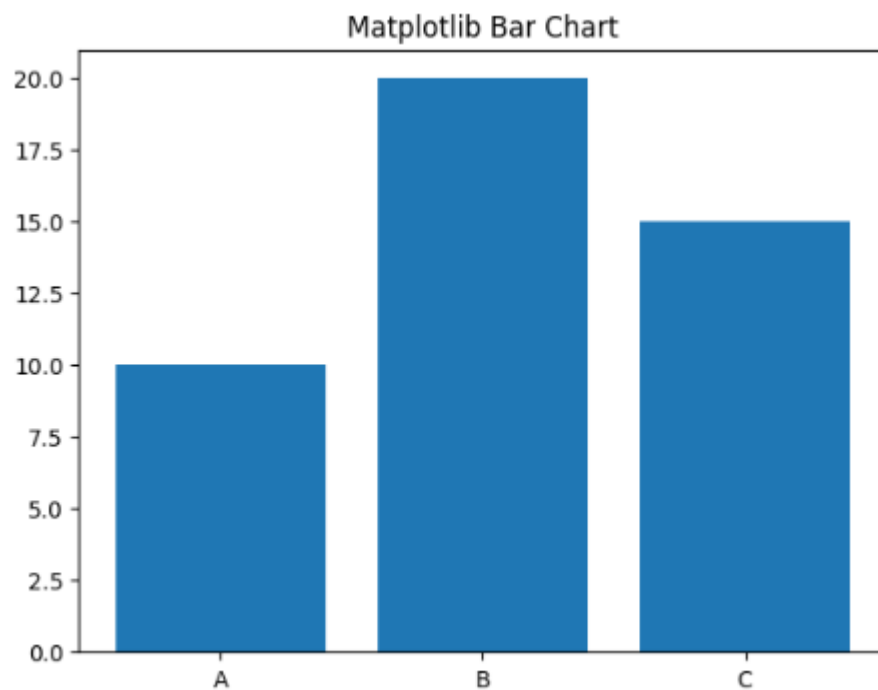
Description: A bar chart uses rectangular bars to compare values across categories.

Use Case: Comparing revenue by product, number of users by country, etc.

```
import matplotlib.pyplot as plt

categories = ['A', 'B', 'C']
values = [10, 20, 15]

plt.bar(categories, values)
plt.title('Matplotlib Bar Chart')
plt.show()
```



4. Histogram

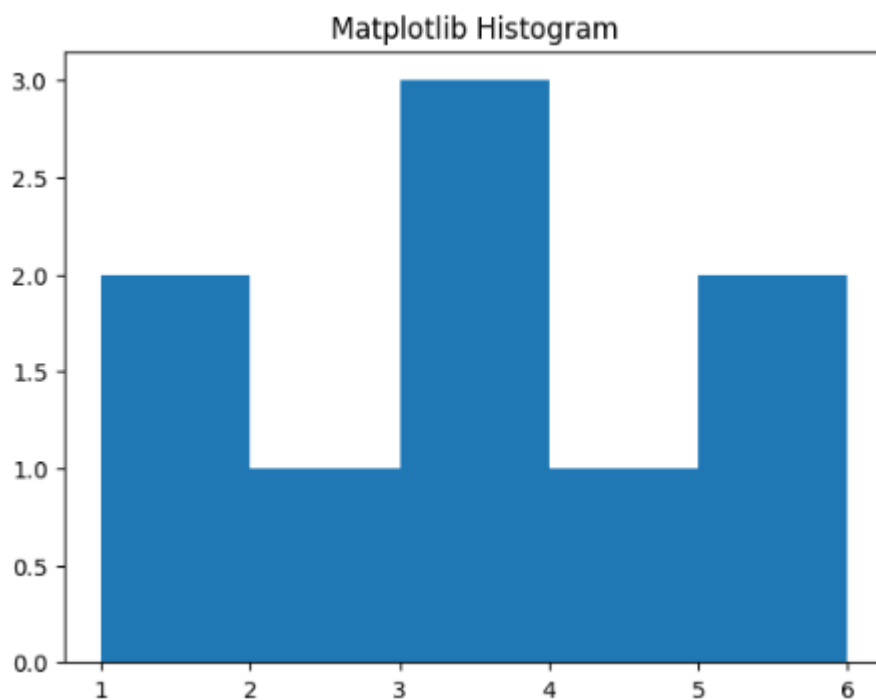
Description: A histogram shows the distribution of a dataset by dividing it into bins and counting the number of values in each.

Use Case: Understanding the spread of exam scores, age distributions, etc.

```
import matplotlib.pyplot as plt

data = [1, 1, 2, 3, 3, 3, 4, 5, 6]

plt.hist(data, bins=5)
plt.title('Matplotlib Histogram')
plt.show()
```



5. Pie Chart

Description: A pie chart displays proportions as slices of a circle. Each slice's size represents the percentage of the whole.

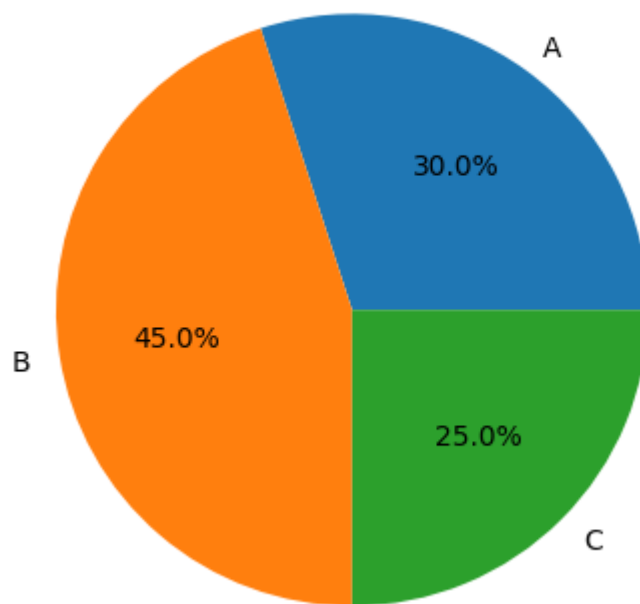
Use Case: Showing percentage of users by platform (mobile, desktop, tablet).

```
import matplotlib.pyplot as plt

labels = ['A', 'B', 'C']
sizes = [30, 45, 25]

plt.pie(sizes, labels=labels, autopct='%1.1f%%')
plt.title('Matplotlib Pie Chart')
plt.show()
```

Matplotlib Pie Chart



Plotly

Library Overview

Plotly is a modern, interactive graphing library for web-based and dashboard-ready visualizations. It allows users to create rich, interactive plots that respond to mouse hovers, zooms, and clicks—ideal for dashboards and data apps.

Use Cases: Interactive dashboards, web apps, exploratory analysis.

Key Features:

- Interactive by default
- Built-in tooltips, zooming, panning
- Integrates with Dash for building dashboards
- Exports to HTML, PNG, and static images

Graph Types and Examples

1. Line Plot

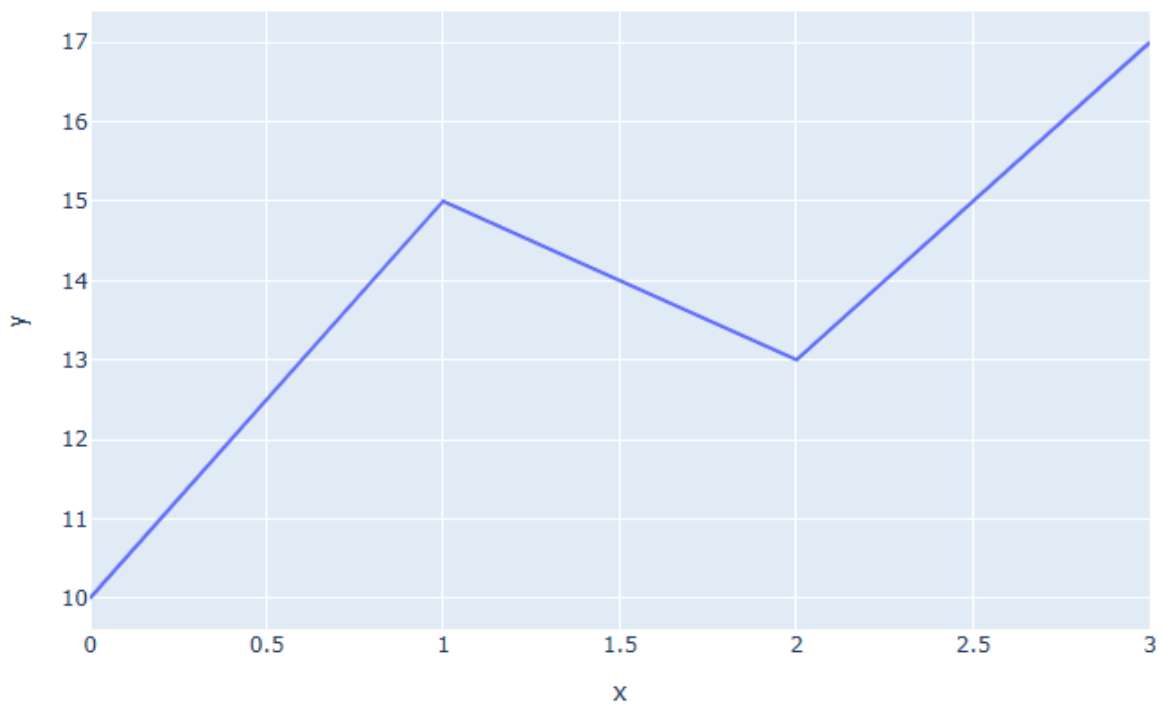
Description: A line plot connects data points with a continuous line. It's used to show trends over time or ordered categories.

Use Case: Tracking stock prices, temperature changes over time, or performance metrics.

```
import plotly.express as px
import pandas as pd

df = pd.DataFrame({'x': [0, 1, 2, 3], 'y': [10, 15, 13, 17]})
fig = px.line(df, x='x', y='y', title='Plotly Line Plot')
fig.show()
```

Plotly Line Plot



2. Scatter Plot

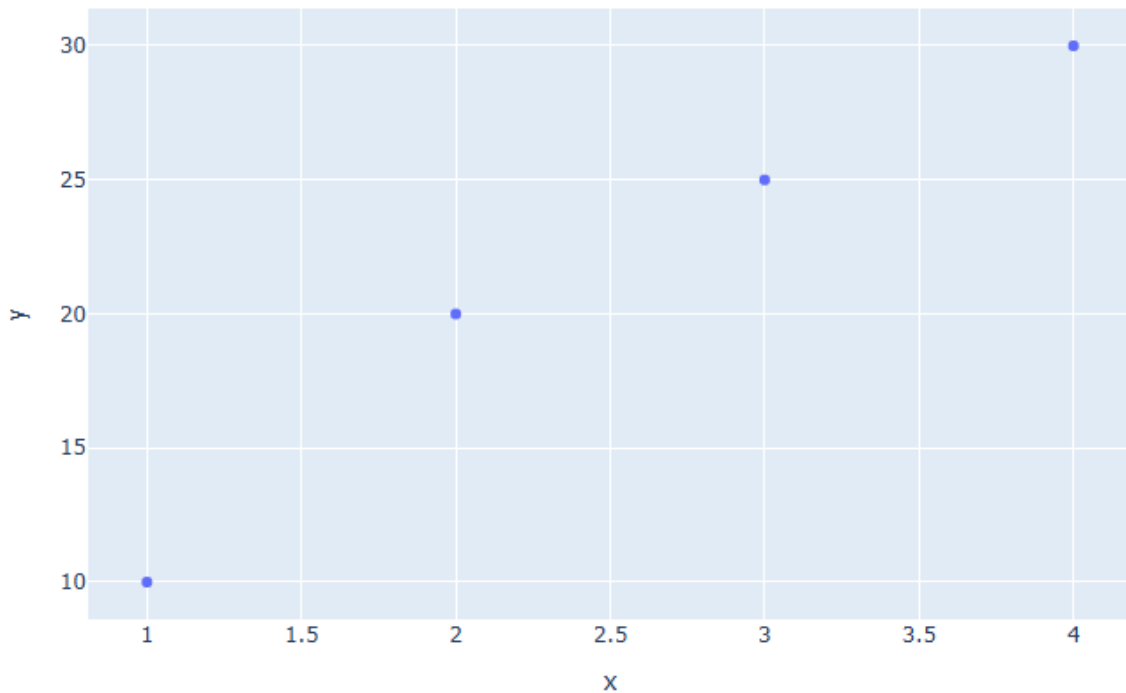
Description: A scatter plot displays points representing the values of two variables. It shows correlation, clustering, and outliers.

Use Case: Visualizing relationships between variables (e.g., height vs weight).

```
import plotly.express as px
import pandas as pd

df = pd.DataFrame({'x': [1, 2, 3, 4], 'y': [10, 20, 25, 30]})
fig = px.scatter(df, x='x', y='y', title='Plotly Scatter Plot')
fig.show()
```


Plotly Scatter Plot



3. Bar Chart

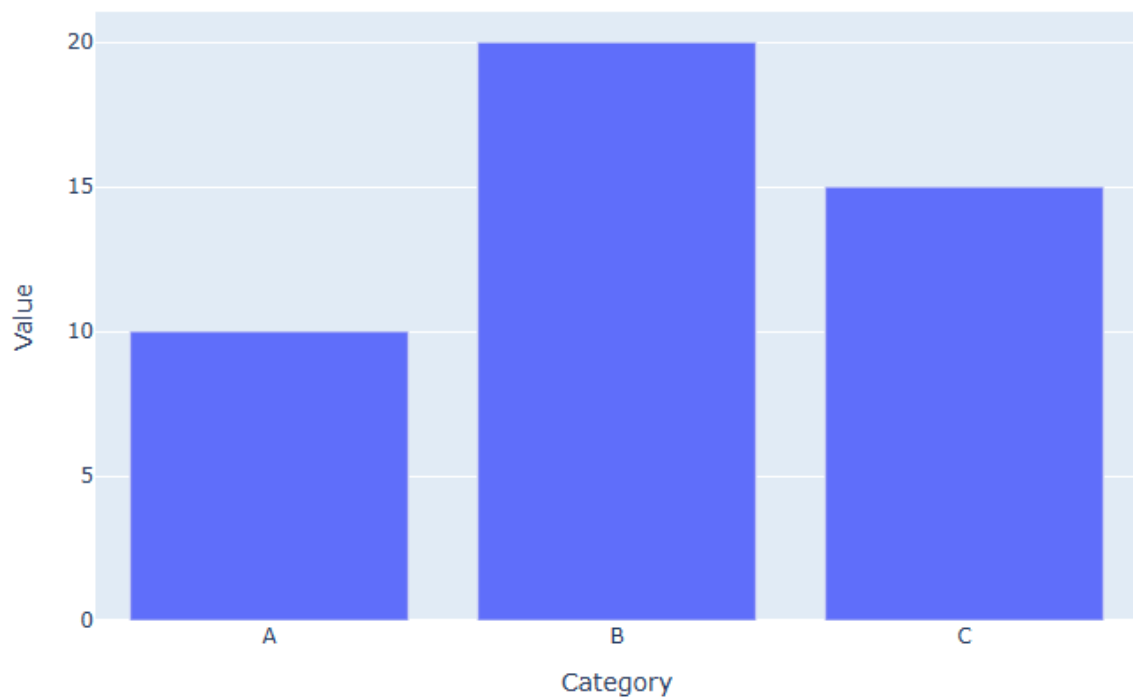
Description: A bar chart uses rectangular bars to compare values across categories.

Use Case: Comparing revenue by product, number of users by country, etc.

```
import plotly.express as px
import pandas as pd

df = pd.DataFrame({'Category': ['A', 'B', 'C'], 'Value': [10, 20, 15]})
fig = px.bar(df, x='Category', y='Value', title='Plotly Bar Chart')
fig.show()
```

Plotly Bar Chart



4. Histogram

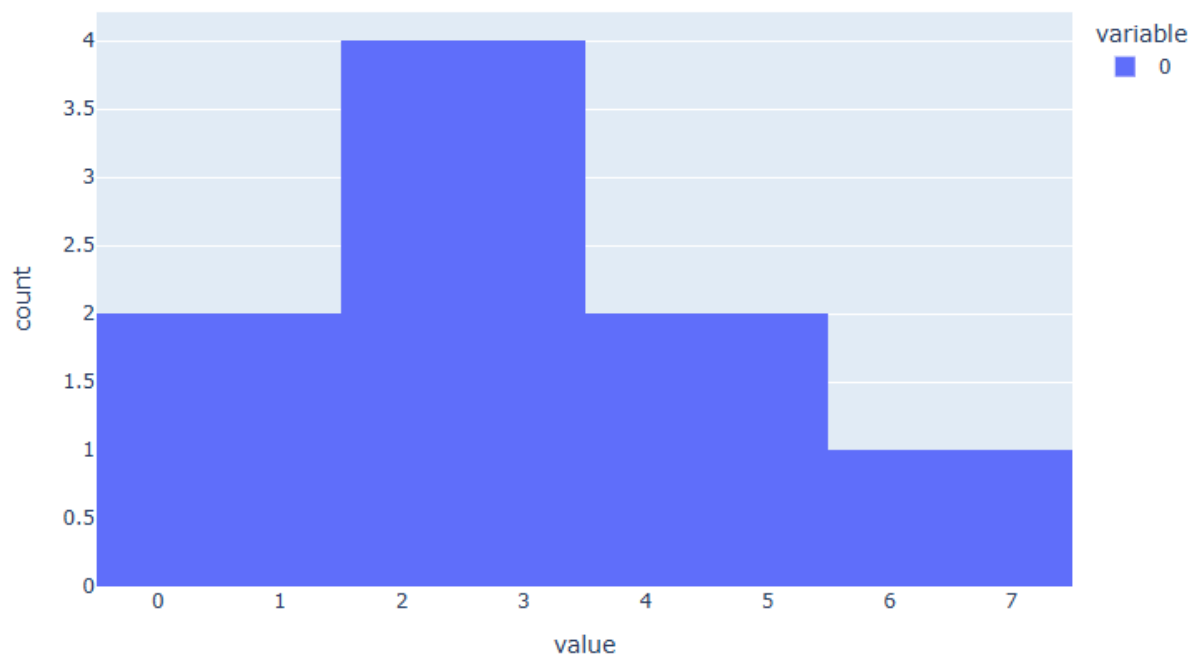
Description: A histogram shows the distribution of a dataset by dividing it into bins and counting the number of values in each.

Use Case: Understanding the spread of exam scores, age distributions, etc.

```
import plotly.express as px
import pandas as pd

fig = px.histogram(data, nbins=5, title='Plotly Histogram')
fig.show()
```

Plotly Histogram



5. Pie Chart

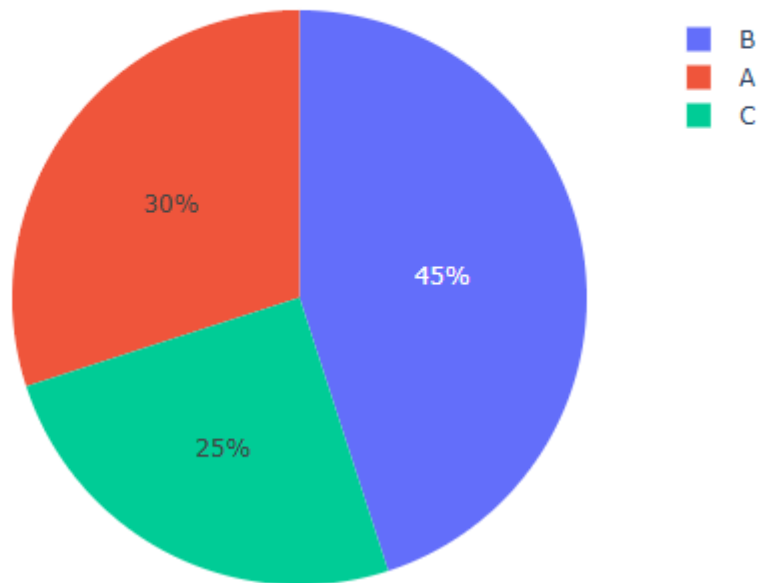
Description: A pie chart displays proportions as slices of a circle. Each slice's size represents the percentage of the whole.

Use Case: Showing percentage of users by platform (mobile, desktop, tablet).

```
import plotly.express as px
import pandas as pd

fig = px.pie(values=[30, 45, 25], names=['A', 'B', 'C'], title='Plotly Pie Chart')
fig.show()
```

Plotly Pie Chart



Comparison: Matplotlib vs Plotly

Feature	Matplotlib	Plotly
Ease of Use	Requires more lines of code, especially for layout	Intuitive and concise syntax
Customization	Extremely flexible (fonts, grids, ticks, etc.)	Somewhat limited customization; focus on simplicity
Interactivity	Static plots by default	Fully interactive out of the box
Output Formats	PNG, PDF, SVG, EPS	HTML, PNG, static images, export as web app
Integration	Works well with scientific tools	Works well with web frameworks (Dash, Jupyter, etc.)
Performance	Fast and lightweight for static plots	May lag with very large datasets in full interactivity
Learning Curve	Steeper due to manual control	Easy for beginners with clean, high-level API